

KONERU LAKSHMAIAH EDUCATION FOUNDATION
(Deemed to be University)

**DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND
DEVELOPMENT (25CS1302E)**

Y25 - 2026-2027 Trimester - 4

PROJECT DOCUMENTATION

SMART RESTAURANT ORDERING SYSTEM

A Project Report submitted as part of the course requirements

Section No: 10 | Team No: 15

Submitted by

Roll Number	Student Name
2520090141 (TL)	R Nishanth
2520090050	Shanmukha T
2520080032	S Gopal

Guide Name: _____

K L (Deemed to be) University
Hyderabad, Telangana

DECLARATION

We hereby declare that the project report entitled “Smart Restaurant Ordering System” is prepared as part of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E). The work documented here is based on the submitted project abstract and the planned technology stack. Sections describing implementation design, database structure, testing, deployment, and appendices are prepared as a draft aligned with the submitted scope and should be updated wherever the final implementation differs.

Team Members:

- 2520090141 (TL) - R Nishanth
- 2520090050 - Shanmukha T
- 2520080032 - S Gopal

Signatures: _____

CERTIFICATE

This is to certify that the project report entitled “Smart Restaurant Ordering System” has been prepared by R Nishanth (2520090141), Shanmukha T (2520090050), and S Gopal (2520080032) for Y25 - 2026-2027 Trimester - 4, Section 10, Team 20, under the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E).

Guide Signature: _____

Course Coordinator Signature: _____

Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to the faculty members and course coordinators of K L (Deemed to be) University for their guidance and support in carrying out this project. We also thank our guide for the technical suggestions and review support provided during the planning and development of the Smart Restaurant Ordering System. We acknowledge the contribution of all team members in requirement analysis, database design, interface development, backend planning, and documentation. Finally, we thank our classmates, friends, and families for their encouragement and cooperation throughout the project work.

ABSTRACT

The Smart Restaurant Ordering System is a database-driven application designed to improve the efficiency and convenience of restaurant ordering and management. Traditional restaurant ordering methods often involve manual order taking, which can result in incorrect orders, delays, difficulty in tracking orders, and increased workload for restaurant staff. Managing menu items, customer orders, billing, and order status manually can also make it difficult to maintain accurate and organized records.

The proposed system provides a centralized platform for managing restaurant operations through a structured database. Customers can view the available menu, select food items, specify quantities, and place orders. The system stores customer, menu, order, and billing information in a database and allows restaurant staff to manage menu items and update order statuses. The system can also calculate the total bill based on the selected items and quantities. Database concepts such as tables, relationships, primary keys, foreign keys, SQL queries, joins, constraints, and transaction handling can be applied to maintain data consistency and efficient retrieval. The system aims to reduce manual effort, minimize ordering errors, improve order processing, and provide an organized approach to restaurant data management.

Keywords: Restaurant ordering, Java, MySQL, JDBC, database management, billing, order status.

DOCUMENTATION STATUS NOTE

This report is a filled project-documentation draft based on the submitted abstract and the listed tools: HTML, CSS, JavaScript, Java, MySQL, JDBC, Visual Studio Code / IntelliJ IDEA, and MySQL Workbench. Where the abstract does not provide implementation evidence (for example, final screenshots, measured performance, executed test results, authentication design, hosting platform, or production APIs), the report uses “proposed”, “planned”, or “to be verified” wording rather than presenting those details as completed facts. Replace those items with the actual implementation before final submission.

INDEX

S.No.	Chapter	Topics
1	Introduction	Background, Problem Statement, Scope, Objectives, Importance, Target Users
2	System Requirements	Hardware, Software, Development Tools and Frameworks
3	Technology Stack	Front-end, Back-end, Database, Connectivity, Development Environment
4	System Architecture	High-level Architecture, Layer Description, Request Flow, Deployment View
5	Design	Use Cases, Database Schema, ER Diagram, Data Dictionary
6	Implementation	Module-wise Implementation, Front-end Logic, JDBC Integration, Transaction Handling
7	Features	Main Features and User/Technical Workflow
8	Testing	Testing Strategy, Test Cases, Expected Results
9	Deployment	Local Deployment, Environment Configuration, Execution Steps
10	Challenges & Limitations	Likely Issues, Mitigations, Current Limitations
11	Future Enhancements	Planned Extensions and Optimizations
12	Conclusion	Summary, Achievements, Skills Learned
13	References	Documentation and learning resources
14	Appendices	Sample SQL, JDBC snippets, setup guide, user guide

CHAPTER 1 - INTRODUCTION

1.1 Background of the Project

Restaurants handle a large number of customer orders every day, and in many restaurants the ordering process is still carried out manually. Customers usually depend on waiters to view the menu, place orders, request quantity changes, receive the bill, and complete payment. This method works reasonably well when the restaurant is less crowded, but during peak hours it can lead to delays, incorrect orders, communication problems, and increased workload for restaurant staff.

The Smart Restaurant Ordering System was developed to simplify this process by providing customers with a digital platform for viewing menu items and placing food orders. Instead of depending completely on manual order-taking, customers can browse the available dishes through a web interface, search for specific items, filter menu items based on their preferences, select required quantities, and place an order directly.

The system stores restaurant menu and order information in a structured database. Menu categories, menu items, orders, and individual order items are maintained separately and connected using relational database concepts. This helps ensure that order information remains organized and easy to retrieve.

The application also performs automatic bill calculation. When the customer confirms an order, the backend calculates the subtotal based on the selected items and quantities and adds the applicable 5% GST to determine the final payable amount.

To make payment easier, the system provides a UPI QR-code payment option. After an order is created, a unique order number and payment information are generated so that the customer can complete the payment digitally.

The project therefore provides a centralized and database-driven restaurant ordering solution that reduces manual effort and improves the accuracy and convenience of food ordering.

1.2 Problem Statement

In a traditional restaurant ordering system, customers depend on restaurant staff for almost every stage of the ordering process. A waiter must provide the menu, record the selected dishes, note quantities, communicate the order to the kitchen, calculate the bill, and collect payment.

This manual process can become inefficient when several customers are being served at the same time. Orders may be written incorrectly, quantities may be misunderstood, or customers may need to wait before a waiter becomes available.

The system must therefore provide a reliable method for answering the following questions in real time:

Which food items are currently available?

What is the price of each menu item?

Which items has the customer selected?

What quantity has been selected for each item?

What is the correct subtotal and final amount after tax?

Has the order been created successfully?

What is the current payment status of the order?

Handling all these tasks manually is time-consuming and error-prone. The Smart Restaurant Ordering System solves this problem by providing a web-based application where menu browsing, cart management, order creation, billing, and payment can be handled digitally.

1.3 Scope of the Project

The system covers the main restaurant-ordering workflow from menu selection to payment.

The project includes:

Displaying restaurant menu items.

Organizing menu items into categories.

Filtering vegetarian and non-vegetarian items.

Filtering food items by meal type.

Searching for food items.

Adding food items to a shopping cart.

Increasing or decreasing item quantities.

Calculating subtotal automatically.

Applying 5% GST.

Generating a unique order number.

Storing order details in the MySQL database.

Storing individual order items separately.

Generating a UPI payment QR code.

Updating payment status.

Communicating between frontend and backend using REST APIs.

The current version focuses mainly on customer-side food ordering and order storage. Advanced features such as inventory tracking, automatic kitchen display systems, customer login, delivery management, real-time payment gateway verification, and online cloud deployment are outside the current scope of the project.

1.4 Objectives

Provide a digital restaurant menu through a simple web interface.

Allow customers to search and filter menu items easily.

Allow customers to add food items to a cart and manage quantities.

Automatically calculate the subtotal and 5% GST.

Store order and menu information in a relational database.

Generate a unique order number for every order.

Provide UPI QR-code-based payment support.

Reduce manual effort in restaurant ordering.

Minimize ordering and billing errors.

Provide a structured and easy-to-maintain restaurant database.

Demonstrate integration of frontend, backend, REST APIs, and MySQL.

1.5 Importance of the Application

A structured ordering system helps create a single source of truth for restaurant data. It can reduce repeated data entry, make bills easier to verify, and provide staff with a clearer view of each order's status. From an academic perspective, the project provides a practical setting for applying relational-database concepts such as primary keys, foreign keys, joins, constraints, transactions, and SQL queries.

1.6 Target Users / Audience

The Smart Restaurant Ordering System mainly serves restaurant customers and restaurant management.

Customers use the system to browse menu items, search for dishes, filter food items, add products to the cart, place orders, view bill amounts, and complete payments.

Restaurant staff and management benefit from the digital storage of order information and the structured organization of menu data.

The main users are:

- Restaurant customers
- Restaurant service staff
- Billing staff
- Restaurant management
- Kitchen staff in future versions

This is the right writing style because your mam's sample also keeps **Chapter 1 compact and project-specific**, rather than writing many pages of general theory. For example, the sample's Chapter 1 directly explains the problem, scope, objectives, importance, and users in about two pages.

CHAPTER 2 - SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Component	Suggested Development Requirement
Processor	Dual-core or better; modern Intel/AMD equivalent
RAM	Minimum 4 GB; 8 GB recommended for IDE + MySQL
Storage	At least 5 GB free for JDK, IDE, MySQL, project files, and backups
Display	1366×768 or higher
Network	Optional for package/documentation access; not required for a fully local build

2.2 Software Requirements

Layer / Purpose	Software
Frontend	HTML, CSS, JavaScript
Backend	Java
Database	MySQL
Database Connectivity	JDBC
Development Environment	Visual Studio Code or IntelliJ IDEA
Database Administration	MySQL Workbench
Java Runtime	JDK 17+ recommended for development
Browser	Any modern browser if the frontend is web-based

2.3 Development Tools and Frameworks

The frontend of the application is developed using **React with TypeScript** and bundled using **Vite**.

The backend is developed using **Node.js and Express.js**, which provide REST API endpoints for menu retrieval, order creation, and payment-related operations.

MySQL is used as the relational database, and the backend connects to MySQL using the **mysql2** package.

Visual Studio Code is used as the main development environment. **MySQL Workbench** is used to inspect tables, execute SQL queries, and verify stored data.

Postman can be used for testing backend REST APIs, while **Git** can be used for source-code version control.

CHAPTER 3 - TECHNOLOGY STACK

3.1 Front-End

The frontend of the Smart Restaurant Ordering System is developed as a single-page web application using **React and TypeScript**.

React is used to build reusable user-interface components such as menu cards, filter controls, cart sections, checkout views, and payment displays. TypeScript provides type checking, which helps reduce programming errors and improves code maintainability.

The frontend is bundled using **Vite**, which provides a fast development environment and quick application startup.

CSS is used for styling the application and making the interface visually clear and responsive.

The frontend handles user interactions such as searching menu items, filtering food categories, adding items to the cart, changing quantities, and displaying the final bill.

3.2 Back-End

The backend is implemented using Node.js and Express.js.

Express.js handles the REST API endpoints used by the frontend. The backend receives requests from the user interface, validates the requested data, communicates with the database, calculates order totals, and sends structured JSON responses back to the frontend.

The backend is also responsible for generating the order number, calculating 5% GST, storing orders, and handling payment status updates.

3.3 Database

MySQL is used as the relational database.

The main database tables include:

menu_categories

menu_items

orders

order_items

The database uses primary keys and foreign keys to maintain proper relationships between menu items and customer orders.

The order_items table stores individual food items belonging to each order, while the orders table stores overall order details such as subtotal, tax, total amount, and status.

The mysql2 Node.js package is used to connect the Express backend with the MySQL database.

3.4 Version Control

Git can be used for version control throughout development.

The entire project can be maintained in a Git repository containing the frontend, backend, and database-related files. Version control helps track changes, maintain previous versions, and avoid accidental loss of project code.

3.5 Supporting Libraries and Services

The project uses supporting packages and browser technologies to implement the required functionality.

The backend uses packages required for Express server execution and MySQL connectivity.

The frontend uses React and TypeScript-based components for interface development.

The application also generates a UPI payment URI and QR code, which provides the customer with a convenient way to initiate digital payment.

CHAPTER 4 - SYSTEM ARCHITECTURE

4.1 High-Level Architecture

The **Smart Restaurant Ordering System – Spice Garden** follows a three-layer architecture consisting of the **Presentation Layer, Application Layer, and Data Layer**. This structure separates the user interface, business logic, and database operations, making the application easier to understand, maintain, and extend.

The user interacts with the React-based frontend to browse menu items, apply filters, add food items to the cart, place an order, and complete payment. The frontend communicates with the Node.js and Express.js backend using REST API requests. The backend performs validations, calculates the bill, communicates with the MySQL database, generates the order number, and returns the required response to the frontend.

The overall architecture can be represented as:

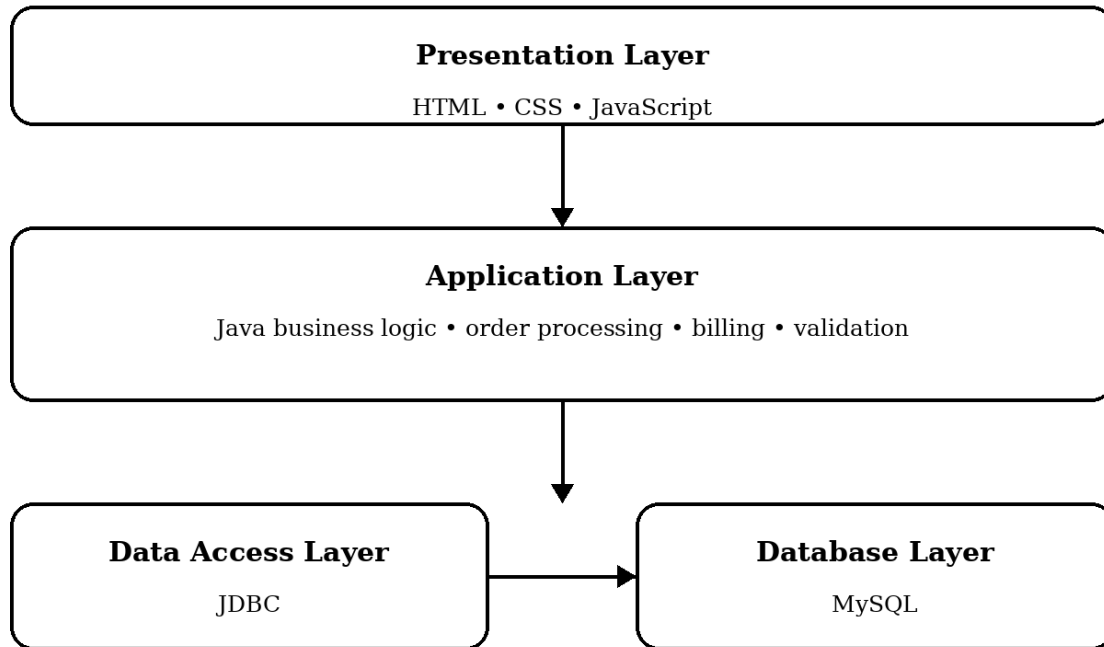


Figure 4.1 Proposed high-level architecture

4.2 Description of Each Layer

Presentation Layer

The Presentation Layer represents the user-facing part of the Smart Restaurant Ordering System. It is developed using React, TypeScript, CSS, and Vite.

This layer provides the interface through which customers interact with the application. The main functions performed in this layer include:

Displaying restaurant menu items.

Searching for food items.

Filtering dishes based on vegetarian or non-vegetarian type.

Filtering menu items based on meal categories.

Adding items to the shopping cart.

Increasing or decreasing item quantities.

Displaying the order subtotal, GST, and final amount.

Displaying the UPI payment QR code.

Showing order and payment status information.

The frontend does not directly access the MySQL database. Instead, it communicates with the backend through REST API calls. This separation improves security and ensures that all important calculations and database operations are handled on the server side.

Application / Business Logic Layer

The Application Layer is developed using Node.js and Express.js. It acts as the connection between the frontend and the database.

The Express server exposes REST API endpoints under the /api route. These APIs handle operations such as retrieving menu data, creating customer orders, checking database availability, and updating payment information.

The backend performs important business logic before storing any information in the database. When a customer places an order, the backend validates whether the selected items exist and whether the requested quantity is valid.

Instead of trusting prices sent from the frontend, the server retrieves the actual prices of selected menu items from the database. It then calculates the subtotal based on the item price and quantity.

The tax is calculated as:

$$\text{GST} = \text{Subtotal} \times 5 / 100$$

The final order amount is calculated as:

$$\text{Total Amount} = \text{Subtotal} + \text{GST}$$

After calculation, the backend creates an order and generates a unique order number in the following format:

VIR-0001, VIR-0002, VIR-0003, ...

The backend also generates a UPI payment URI containing the restaurant payment information, order number, and total amount. This URI is used by the frontend to display the QR code for payment.

Another important feature of the backend is transaction handling. While creating an order, the main order record and its individual order-item records are stored within a database transaction. If an error occurs during the process, the transaction is rolled back so that incomplete order information is not stored.

Data Layer

The Data Layer consists of the MySQL relational database.

The backend connects to MySQL using the mysql2 package with a connection pool. A connection pool allows multiple database operations to be handled efficiently without creating a completely new database connection for every request.

The major database tables used by the application include:

menu_categories

menu_items

orders

order_items

The menu_categories table stores food-category information.

The menu_items table stores information about individual dishes, such as item name, description, price, food type, meal type, category, and availability.

The orders table stores the overall details of each customer order, including order number, table number, subtotal, tax, total amount, order status, and payment status.

The order_items table stores the individual dishes included in each order along with their quantities and prices.

Primary keys and foreign keys are used to establish relationships between these tables and maintain data consistency.

4.3 Deployment Architecture

For development and classroom demonstration, the Smart Restaurant Ordering System can run on a single computer.

The application consists of three main components:

React/Vite Frontend

Node.js/Express Backend

MySQL Database

During development, the frontend is executed using the Vite development server. The Express backend runs separately and processes REST API requests.

The MySQL database normally runs on its default port 3306. The backend server uses environment variables to obtain database configuration details such as database host, username, password, database name, and port.

The Express server uses the port specified in the environment configuration. If a port is not explicitly provided, the application uses port 3000.

The backend also serves the compiled frontend files when the application is executed in production mode. This allows the complete application to be built and served through the Node.js server.

A simplified deployment flow is:

User Browser

↓

React Web Application

↓

Express REST API

↓

MySQL Database

An additional reliability mechanism is provided for development and demonstration. If the MySQL database is temporarily unavailable, the application can use predefined menu data and temporary in-memory order storage. This allows basic menu browsing and order-processing functions to continue during local testing.

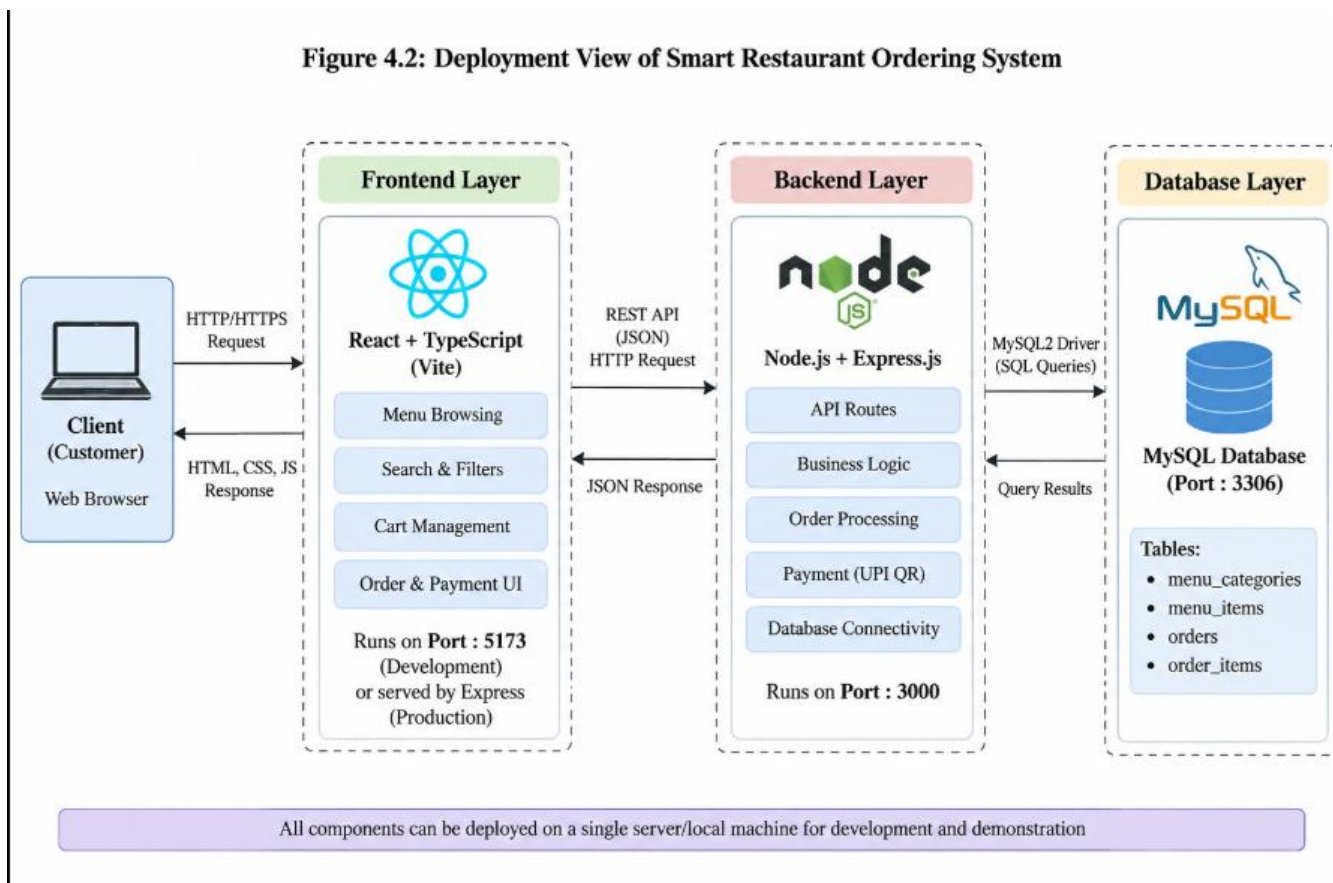
For the current academic project, the application is primarily designed for local development and demonstration. Cloud deployment, Docker containerization, load balancing, and automated CI/CD deployment can be considered as future enhancements.

This chapter should occupy roughly one page like your mam's Chapter 4, with the architecture diagram placed between 4.1 and 4.2, similar to the sample. Final documentation (1)

Figure 4.2 Proposed order-processing flow

4.4 Deployment View

For a classroom or laboratory deployment, the application can run on a local development machine with the Java application and MySQL server installed together. A networked deployment could separate the database and application server, but no final hosting platform is specified in the submitted project abstract.



CHAPTER 5 - DESIGN

5.1 Database Design Principles

The database of the Smart Restaurant Ordering System is designed using a relational model so that menu information and customer order details can be stored in a structured and consistent manner.

The database used for the project is named `viraham_restaurant` and is implemented using MySQL. The schema is divided into four main tables: `menu_categories`, `menu_items`, `orders`, and `order_items`.

The design avoids unnecessary duplication by storing each type of information in a separate table. For example, category information is stored only in the `menu_categories` table, while individual food details are stored in the `menu_items` table. Similarly, the overall order information is stored in the `orders` table and the food items belonging to that order are stored separately in the `order_items` table.

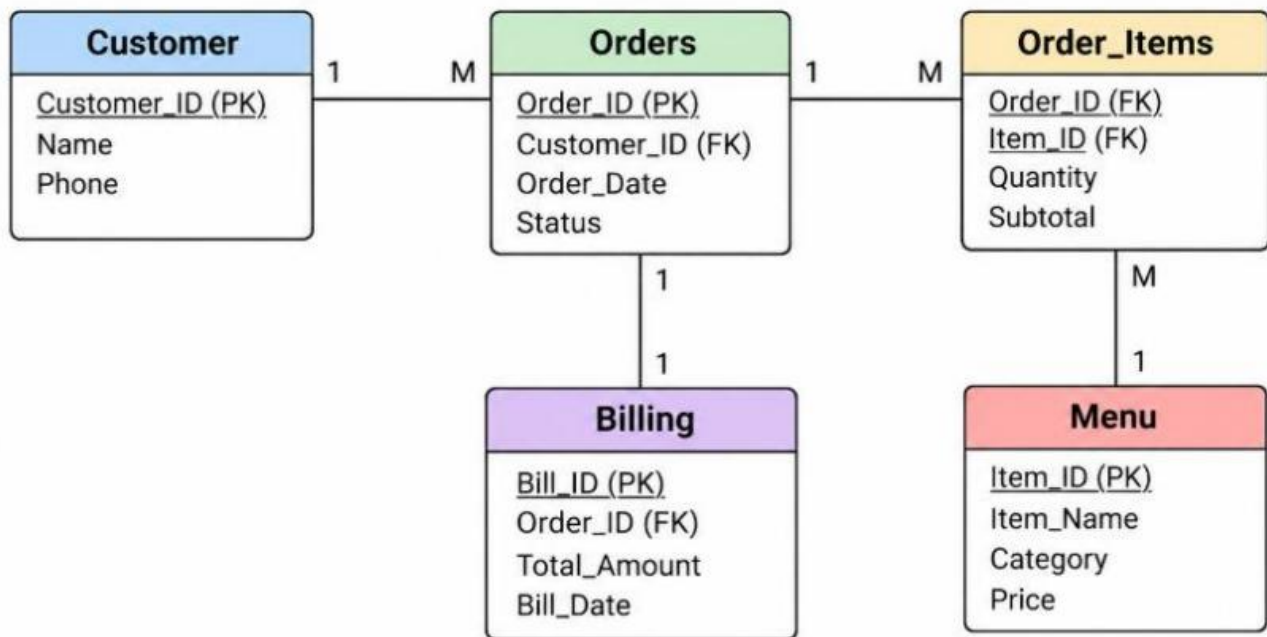
Primary keys are used to uniquely identify each record, while foreign keys establish relationships between related tables. The database also uses constraints such as NOT NULL, UNIQUE, ENUM, and foreign-key constraints to improve data consistency.

The schema uses the InnoDB storage engine, which supports foreign-key relationships and transaction-based operations.

5.2 ER Diagram / Database Schema

The abstract explicitly identifies customer, menu, order, and billing information. The proposed schema below adds an `ORDER_ITEM` junction table so that one order can contain multiple menu items and one menu item can appear in many orders.

ER Diagram



The main attributes are:

- category_id – Primary key used to uniquely identify a category.
- category_name – Stores the name of the category and must be unique.
- display_order – Specifies the order in which categories should be displayed.

One menu category can contain multiple menu items. Therefore, the relationship between menu_categories and menu_items is **one-to-many**.

MENU_ITEMS

The menu_items table stores information about each food item available in the restaurant.

The main attributes are:

- item_id – Primary key.
- category_id – Foreign key referencing menu_categories.
- name – Name of the food item.
- description – Description of the dish.
- price – Price of the menu item.

- `food_type` – Specifies whether the item is VEG or NON_VEG.
- `meal_type` – Specifies whether the item belongs to BREAKFAST, LUNCH, SNACKS, DINNER, DRINKS, or SWEETS.
- `is_available` – Indicates whether the food item is currently available.
- `image_url` – Stores the image reference for the menu item.
- `created_at` – Stores the time when the item was created.

Each menu item belongs to one category, while one category may contain many menu items.

ORDERS

The orders table stores the overall details of every customer order.

The main attributes are:

- `order_id` – Primary key.
- `order_number` – Unique order number generated for each order.
- `table_number` – Stores the customer's table number.
- `subtotal` – Total price of selected items before tax.
- `tax` – Stores the calculated GST amount.
- `total` – Stores the final payable amount.
- `status` – Indicates the current state of the order.
- `payment_status` – Indicates the current payment state.
- `created_at` – Stores the date and time when the order was created.

The order status can take values such as:

PLACED, PREPARING, READY, COMPLETED, and CANCELLED.

The payment status can take values such as:

PENDING_PAYMENT, PAID, and PAYMENT_FAILED.

ORDER_ITEMS

The order_items table stores each individual menu item that belongs to an order.

Its main attributes are:

- `order_item_id` – Primary key.
- `order_id` – Foreign key referencing the orders table.
- `item_id` – Foreign key referencing the menu_items table.

- quantity – Number of units ordered.
- price – Price of the item at the time of ordering.

One order can contain multiple order items. Therefore, the relationship between orders and order_items is **one-to-many**.

Similarly, one menu item can appear in several different orders through the order_items table.

Main Relationships

The important relationships in the database are:

MENU_CATEGORIES (1) → (M) MENU_ITEMS

One category can contain many food items.

ORDERS (1) → (M) ORDER_ITEMS

One customer order can contain multiple food items.

MENU_ITEMS (1) → (M) ORDER_ITEMS

One menu item can occur in many different customer orders.

These relationships allow the system to store order information without repeatedly duplicating complete menu-item details.

Figure 5.1 Proposed ER/database relationship diagram

5.3 Design Decisions and Assumptions

Several design decisions were made while developing the Smart Restaurant Ordering System.

Separation of Orders and Order Items

The overall order information and the individual food items are maintained in separate tables.

The orders table stores common information such as the order number, table number, subtotal, tax, total amount, order status, and payment status.

The order_items table stores the individual items and quantities belonging to the order.

This structure allows a single order to contain any number of food items without duplicating the main order information.

Menu Category Separation

Menu categories are stored in a separate menu_categories table instead of storing repeated category names for every item.

Each menu item refers to its category through category_id. This reduces duplicate information and makes category management easier.

Controlled Food and Meal Types

The database uses ENUM values for food type and meal type.

Food type is restricted to:

VEG

NON_VEG

Meal type is restricted to:

BREAKFAST

LUNCH

SNACKS

DINNER

DRINKS

SWEETS

This prevents invalid category values from being entered into the database.

Order Status Management

The application uses predefined order statuses to represent the lifecycle of an order:

PLACED → PREPARING → READY → COMPLETED

An order may also be marked as CANCELLED when required.

This design makes it easier to extend the application later with kitchen and restaurant staff dashboards.

Payment Status Management

Payment information is maintained separately from the food-preparation status.

An order may have one of the following payment states:

PENDING_PAYMENT

PAID

PAYMENT_FAILED

Keeping order status and payment status separate makes the system more flexible because payment completion and food preparation are two different processes.

Referential Integrity

Foreign keys are used to maintain valid relationships between the tables.

If a menu category is deleted, its associated menu items are configured to be deleted using ON DELETE CASCADE.

Similarly, if an order is deleted, its corresponding records in order_items are also automatically removed.

This prevents orphan records from remaining in the database.

CHAPTER 6 - IMPLEMENTATION

6.1 Module-Wise Implementation

The Smart Restaurant Ordering System is implemented as a full-stack web application with separate frontend, backend, and database components. The major functional modules are Menu Management, Search and Filtering, Cart Management, Order Processing, Payment Processing, and Order Status Management.

Menu Module

The menu module is responsible for displaying all available food items to the customer. Menu data is retrieved from the backend using the GET /api/menu endpoint.

Only items marked as available are returned from the MySQL database. Each menu item contains details such as item name, description, price, food type, category, and meal type.

The backend also supports filtering parameters such as food_type and meal_type. This allows the frontend to request only vegetarian, non-vegetarian, breakfast, lunch, snacks, dinner, drinks, or sweets items when required.

If the MySQL database is temporarily unavailable during local testing, the application uses predefined fallback menu data so that the menu can still be displayed.

Search and Filter Module

The frontend provides two main types of filtering.

The first filter is based on food preference:

All

Veg

Non-Veg

The second filter is based on meal category:

All

Breakfast

Lunch

Snacks

Dinner

Drinks

Sweets

A text-based search facility is also implemented. The search checks the item name, description, and category and displays only the matching food items.

This allows customers to find required dishes quickly without manually browsing the entire menu.

Cart Management Module

The cart module allows customers to select food items before placing an order.

When the user clicks the Add button, the selected food item is added to the cart. If the same item is selected again, its quantity is increased instead of adding a duplicate entry.

Inside the cart, the customer can:

View selected food items.

Increase item quantity.

Decrease item quantity.

Remove an item by reducing its quantity to zero.

View the price of each item.

View the subtotal.

View 5% GST.

View the estimated final total.

The subtotal is calculated using:

$$\text{Subtotal} = \Sigma (\text{Item Price} \times \text{Quantity})$$

The GST amount is calculated using:

$$\text{GST} = \text{Subtotal} \times 5\%$$

The total amount is:

$$\text{Total} = \text{Subtotal} + \text{GST}$$

6.2 Front-End Logic

The frontend is implemented using **React and TypeScript**.

The main restaurant page maintains application state using React hooks such as `useState`, `useEffect`, and `useMemo`.

useState is used to maintain information such as:

- Menu items
- Selected food type
- Selected meal category
- Search query
- Shopping cart
- Cart visibility
- Order-processing state
- Pending payment order
- Confirmed order

The useEffect hook is used to fetch menu data from the backend whenever the food-type or category filter changes.

The useMemo hook is used for search filtering so that the displayed list is recalculated based on the current menu data and search text.

The frontend is divided into reusable interface components such as food cards, cart drawer, payment modal, and order confirmation screen.

When the customer adds an item, the cart state is updated immediately. Quantity controls allow the number of selected items to be increased or decreased dynamically without reloading the page.

After the customer clicks **Proceed to Checkout**, the frontend converts the cart information into a simplified list containing item IDs and quantities and sends it to the backend.

The frontend does not calculate or send trusted item prices to the server. This is an important design decision because the backend retrieves the actual price from the database before calculating the final bill.

6.3 JDBC Integration

Communication between the frontend and backend is handled through REST APIs.

A separate API service file is used on the frontend to centralize HTTP operations. This prevents API request logic from being repeated throughout different UI components.

The main API operations implemented in the project are:

Method	Endpoint	Purpose
GET	/api/health	Checks server and database connectivity
GET	/api/menu	Retrieves available menu items
POST	/api/orders	Creates a new customer order
GET	/api/orders/:id	Retrieves an existing order

Method	Endpoint	Purpose
PATCH	/api/orders/:id/payment	Updates payment status
PATCH	/api/orders/:id/status	Updates order status

Order Creation API

When the customer confirms the cart, the frontend sends a POST request to /api/orders.

The backend first verifies that:

The order contains at least one item.

Each item has a valid ID.

Each item has a quantity greater than zero.

The requested item exists in the database.

The selected item is currently available.

The backend then retrieves the actual item prices from MySQL. This prevents the frontend from modifying the price before the order is created.

After validation, the subtotal, GST, and total amount are calculated.

Database Transaction

Order creation uses a MySQL transaction.

First, a new row is inserted into the orders table. The generated database ID is then used to create a formatted order number such as:

VIR-0001

After that, every selected food item is inserted into the order_items table.

If all operations are completed successfully, the transaction is committed. If any database operation fails, the transaction is rolled back.

This prevents partially created orders from remaining in the database.

6.4 Payment and Order Status Handling

After successfully creating an order, the backend generates a UPI payment URI containing the restaurant UPI ID, restaurant name, total amount, and order number.

The frontend uses this URI to generate a QR code through the payment modal.

The payment screen displays:

Order number

Table number

Ordered items

Subtotal

GST

Total amount

UPI QR code

The customer can scan the QR code using a compatible UPI application.

After payment, the customer selects the “I HAVE PAID” option. The frontend sends a PATCH request to:

`/api/orders/:id/payment`

The current implementation then updates the order's payment status to PAID.

The supported payment-status values are:

PENDING_PAYMENT

PAID

PAYMENT_FAILED

The backend also provides an API for updating the restaurant order status. The supported order states are:

PLACED → PREPARING → READY → COMPLETED

The order can also be marked as:

CANCELLED

This structure makes it possible to extend the project in the future with a kitchen or restaurant-management dashboard.

Fallback Handling

An additional implementation feature is the fallback mechanism used when MySQL is unavailable.

If the database connection cannot be established, the application can retrieve menu information from predefined local menu data. Orders created in this mode are temporarily stored in application memory.

This feature is mainly intended for local testing and demonstration and does not replace permanent MySQL storage.

CHAPTER 7 - FEATURES

7.1 List of Main Features

The Smart Restaurant Ordering System provides a set of customer-focused features that make restaurant ordering simple, fast, and organized.

Customer-facing Features

View all available menu items.

Search food items by name, description, or category.

Filter dishes as Veg or Non-Veg.

Filter menu items based on meal type such as Breakfast, Lunch, Snacks, Dinner, Drinks, and Sweets.

Add food items to the cart.

Increase or decrease item quantity.

Remove items from the cart.

View subtotal, GST, and final bill amount.

Enter table number while placing the order.

Generate a unique order number after successful order creation.

View UPI QR code for digital payment.

Confirm payment after completing the transaction.

View order confirmation details.

Backend and Database Features

Retrieve menu items from the MySQL database.

Validate menu item IDs and quantities before order creation.

Retrieve actual item prices from the database.

Automatically calculate 5% GST.

Store order details in the orders table.

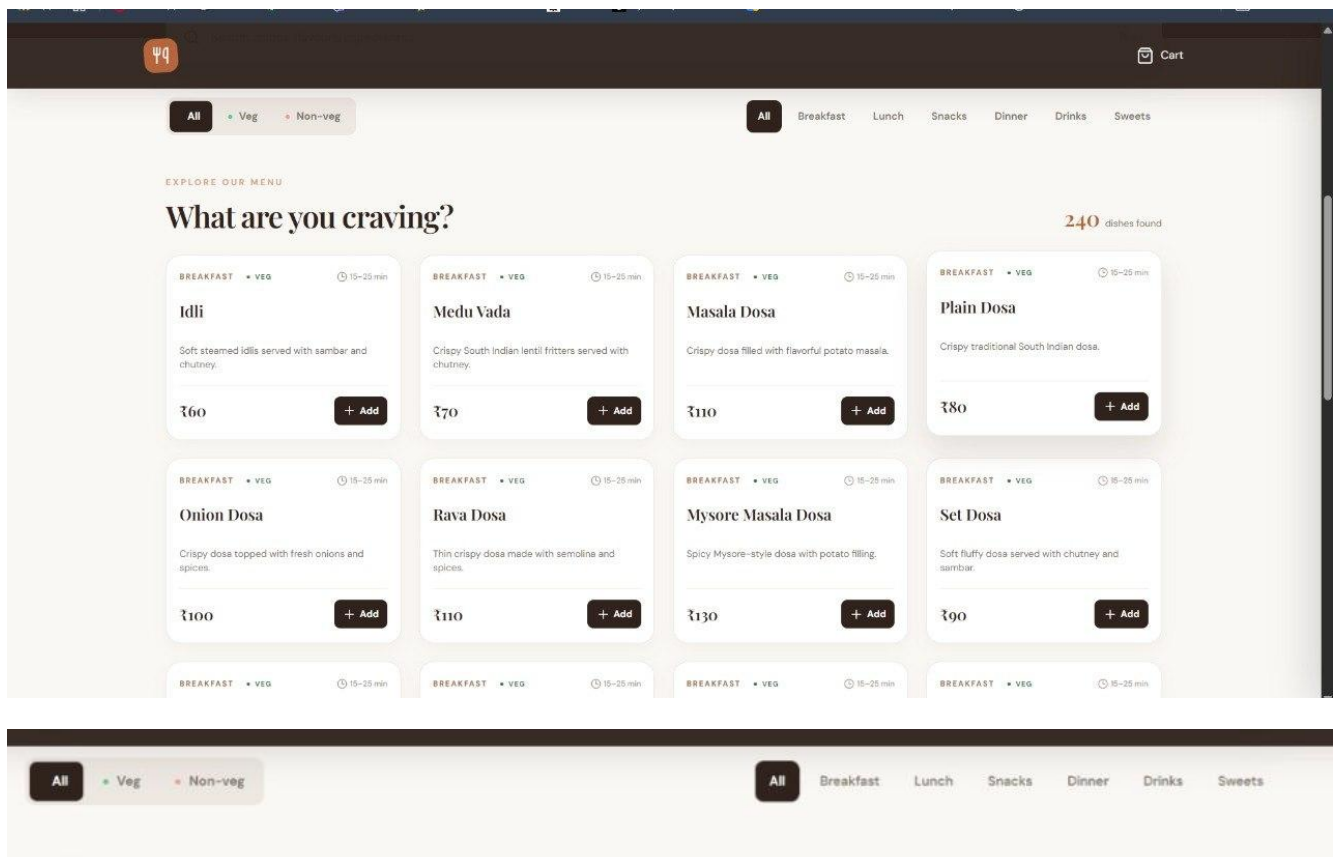
Store individual food items in the order_items table.

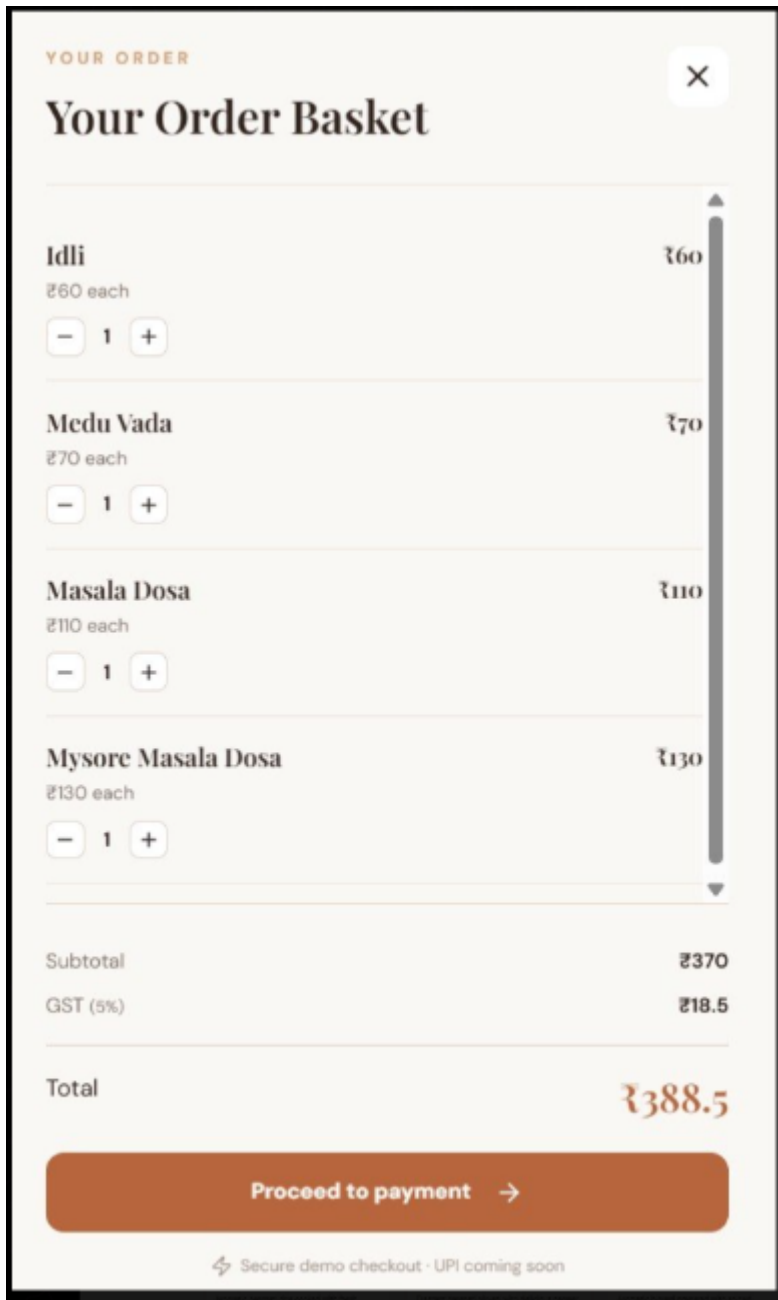
Maintain separate order status and payment status.

Use database transactions while creating orders.

Support fallback menu data and temporary in-memory orders when MySQL is unavailable during testing.

7.2 How the Features Work





Menu Browsing

When the application is opened, the frontend requests the available menu items from the backend.

The backend retrieves the menu information from the database and returns it to the frontend in JSON format. Each menu item contains details such as the food name, description, price, food type, meal type, and category.

The customer can then browse the available food items from the main restaurant page.

Search Functionality

The application includes a search bar that allows customers to quickly find a particular food item.

When the user types text into the search field, the frontend compares the entered text with the menu item's name, description, and category.

Only matching items are displayed on the screen. This allows users to locate dishes quickly without manually browsing through the complete menu.

Food Type Filtering

Customers can filter dishes according to their food preference.

The available options include:

- All
- Veg
- Non-Veg

When a filter is selected, the displayed menu is updated so that only matching food items are shown.

This feature is useful for customers who want to quickly identify vegetarian or non-vegetarian dishes.

Meal Type Filtering

The restaurant menu is also organized using meal-based categories.

The supported meal types include:

- Breakfast
- Lunch
- Snacks
- Dinner
- Drinks
- Sweets

Customers can select a meal category to view only the dishes belonging to that section.

Add to Cart

Each menu item provides an option to add the dish to the shopping cart.

When the customer selects a food item, the application stores the item in the cart together with its quantity.

If the same item is added again, the quantity of the existing cart item is increased instead of creating a duplicate entry.

Cart Management

The cart displays all the dishes selected by the customer.

The customer can:

- Increase quantity.

- Decrease quantity.
- Remove an item.
- View the price of each item.
- View the current subtotal.

Whenever a quantity changes, the bill amount is recalculated automatically.

Automatic Bill Calculation

The system automatically calculates the cost of the customer's order.

The subtotal is calculated as:

Subtotal = Sum of (Item Price × Quantity)

The system then applies **5% GST**.

GST = Subtotal × 0.05

The final amount is:

Total Amount = Subtotal + GST

This reduces the possibility of manual billing errors.

Order Creation

After verifying the selected dishes, the customer proceeds to checkout.

The frontend sends the selected item IDs and quantities to the backend. The backend then validates the order and retrieves the actual prices from the database.

After successful validation and calculation, a new order is inserted into the database.

The system generates a unique order number such as:

The individual dishes belonging to the order are then stored in the order_items table.

UPI QR Code Payment

After the order is successfully created, the application displays a payment screen containing a UPI QR code.

The QR code represents payment information such as the UPI address, restaurant name, total order amount, and order number.

The customer can scan the QR code using a supported UPI payment application.

Payment Confirmation

After completing the payment, the customer can confirm that payment has been made.

The frontend sends the payment update request to the backend, and the payment status of the order is updated.

The payment status can be maintained as:

- PENDING_PAYMENT
- PAID
- PAYMENT_FAILED

Once the payment is confirmed, the application displays the order confirmation information.

Order Status Tracking

The backend is designed to maintain the current status of each order.

The supported order states are:

PLACED → PREPARING → READY → COMPLETED

An order can also be marked as:

CANCELLED

This feature provides the foundation for a future kitchen or restaurant-management dashboard where restaurant staff can update the progress of customer orders.

Database Fallback Support

During local development, the system checks whether the MySQL database is available.

If the database cannot be reached, predefined menu data can be used so that the main interface continues to function. Orders can also be temporarily maintained in memory for demonstration purposes.

This feature improves the reliability of local testing, although permanent order storage requires MySQL.

CHAPTER 8 - TESTING

Executed test results were not included in the submitted abstract. The following is a test plan to be executed against the final build. Replace the “To be verified” entries with actual pass/fail results before final submission.

8.1 Testing Strategy

- Functional testing of customer and staff flows.
- Database testing for keys, constraints, and joins.
- JDBC integration testing for connection and query execution.
- Transaction testing to confirm rollback on partial failure.
- Input-validation testing for invalid quantities and missing data.

8.2 Test Cases and Expected Results

ID	Test	Input / Precondition	Expected Result	Status
TC-01	Load available menu	Menu contains active items	Available items and prices are shown	To be verified
TC-02	Place order with valid items	Valid customer, item IDs, qty > 0	Order and order-item rows are created	To be verified
TC-03	Reject zero quantity	Selected item, qty = 0	Validation error; no invalid line stored	To be verified
TC-04	Calculate total	Known item prices and quantities	Total equals sum(price × quantity)	To be verified
TC-05	Update menu price	Staff edits existing item	New price is stored and retrieved	To be verified
TC-06	Update order status	Existing order	Status changes to selected valid value	To be verified
TC-07	Foreign-key integrity	Invalid item/order reference	Database rejects inconsistent child record	To be verified
TC-08	Transaction rollback	Force failure after order header insert	No partial order remains after rollback	To be verified

8.3 Tools Used for Testing

MySQL Workbench can be used to execute SQL queries and inspect stored data. Java IDE debugging tools can be used for backend execution. Browser developer tools can be used if the frontend is web-based. No dedicated automated-testing library was specified in the submitted abstract.

CHAPTER 9 - DEPLOYMENT

9.1 Local Deployment Steps

1. Install a compatible Java Development Kit (JDK).
2. Install MySQL Server and MySQL Workbench.
3. Create the project database and tables using the SQL schema.
4. Configure the JDBC URL, database username, and password in the Java application.
5. Add the MySQL JDBC driver to the project dependencies/classpath.
6. Run the Java backend or application entry point from IntelliJ IDEA or Visual Studio Code.
7. Open the front-end entry page or application interface and verify database connectivity.
8. Create a sample menu item, place a sample order, and verify that the order and billing data are stored correctly.

9.2 Environment Configuration

Setting	Example / Guidance
JDBC URL	<code>jdbc:mysql://localhost:3306/restaurant_db</code>
Database User	Use a dedicated local project user where possible
Database Password	Store outside source code for a production-ready build
JDBC Driver	MySQL Connector/J
Character Set	utf8mb4 recommended
Timezone	Configure consistently between Java and MySQL if timestamps are used

9.3 Hosting Status

No production hosting platform is specified in the source abstract. This report therefore documents a local deployment path only. If the project is later hosted on a server, add the actual server address, deployment architecture, environment variables, and database-host details here.

CHAPTER 10 - CHALLENGES & LIMITATIONS

10.1 Likely Development Challenges

Challenge	Why It Matters	Recommended Approach
Database consistency	An order spans multiple related tables.	Use foreign keys and transactions.
Correct billing	Client-side values may be changed or stale.	Recalculate totals on the Java backend using database prices.
Concurrent updates	Multiple staff actions can affect the same order/menu.	Use transactions and careful update conditions.
Input errors	Invalid quantities or missing fields can produce bad data.	Validate on both frontend and backend.
Connection failures	JDBC/database errors can interrupt ordering.	Handle exceptions and display clear error feedback.

10.2 Current Limitations

Based strictly on the submitted abstract, the documented system focuses on core ordering and data management. The source does not confirm online payment, delivery tracking, reservation management, kitchen-display integration, mobile apps, analytics, or cloud deployment. These should not be presented as implemented unless they are present in the final project.

10.3 Known Bugs

No known-bug list was supplied. Add observed issues from the final implementation here, including reproduction steps and current workaround, before the final report is submitted.

CHAPTER 11 - FUTURE ENHANCEMENTS

The following enhancements are suggestions rather than claims about the current implementation:

- Customer and staff authentication with role-based access.
- QR-code table ordering so customers can open the menu from their phones.
- Online payment integration and digital receipts.
- Kitchen-display dashboard with real-time order queues.
- Table reservation and dine-in table management.
- Inventory tracking linked to menu-item availability.
- Sales analytics, popular-item reports, and daily revenue summaries.
- Email/SMS/WhatsApp order notifications.
- Cloud deployment with database backup and monitoring.
- Responsive mobile-first interface or dedicated mobile application.

11.1 Customer and Staff Authentication

A login system can be added for customers and staff members. Role-based access can ensure that customers can place and view orders while authorized staff can manage menu items and update order statuses.

11.2 QR-Code Table Ordering

QR codes can be provided for individual restaurant tables. Customers can scan the QR code using their mobile phones and directly access the restaurant menu and ordering interface.

11.3 Online Payment and Digital Receipts

The system can be enhanced with secure online payment gateway integration. After successful payment verification, customers can receive digital receipts containing order and billing details.

11.4 Kitchen Display Dashboard

A kitchen-display dashboard can be developed to show incoming orders in real time. Kitchen staff can view pending orders and update their status as they move from preparation to completion.

11.5 Table Reservation and Management

Future versions can include table reservation functionality. Customers could check table availability and reserve tables in advance, while staff could manage dine-in table information.

11.6 Inventory Management

Inventory tracking can be connected with menu items so that unavailable dishes can automatically be marked unavailable. This can help restaurant staff monitor ingredients and maintain accurate menu availability.

11.7 Sales Analytics and Reports

The system can generate reports such as daily sales, total revenue, popular food items, and order statistics. These reports can help restaurant management understand ordering patterns.

11.8 Notifications

Email, SMS, or WhatsApp notifications can be added to inform customers about order confirmation, payment status, and order completion.

11.9 Cloud Deployment

The application can be deployed to a cloud platform with database backup, monitoring, and scalable infrastructure. This would allow the system to be accessed remotely and support larger numbers of users.

11.10 Mobile Application

A responsive mobile-first interface or dedicated Android/iOS application can be developed to provide customers with a convenient mobile ordering experience.

CHAPTER 12 - CONCLUSION

12.1 Summary of the Project

The **Smart Restaurant Ordering System** is designed as a database-driven application for organizing restaurant ordering activities. The system provides a structured workflow for menu browsing, food selection, cart management, order creation, billing, and order-status management. Menu, customer, order, and billing information can be maintained in an organized database, reducing the dependence on fragmented manual processes.

12.2 What the Project Aims to Achieve

The project aims to reduce manual effort, minimize ordering and billing errors, improve order processing, and maintain organized restaurant records. The system demonstrates how frontend interaction, backend processing, database management, billing, and order-status handling can be combined to create a structured restaurant-ordering solution.

12.3 Skills Demonstrated

- Relational database modeling and normalization concepts.
- Primary-key and foreign-key relationship design.
- SQL queries, joins, constraints, and transaction handling.
- Java backend programming and JDBC database connectivity.
- Basic front-end development using HTML, CSS, and JavaScript.
- Debugging and database inspection using an IDE and MySQL Workbench.
- Project documentation and modular system design.

Through this project, the team gained practical knowledge of relational database design, tables, primary keys, foreign keys, SQL queries, joins, constraints, and transaction handling. The project also provides experience in backend programming, database connectivity, frontend development, debugging, database inspection, and modular project documentation.

CHAPTER 13 - REFERENCES

- **Project Abstract Submission Form – Smart Restaurant Ordering System, Y25 – 2026–2027** Trimester 4, Section 10.
- **Oracle Java Documentation** – Reference material for Java programming, JDBC concepts, and the java.sql package.
- **MySQL Reference Manual** – Reference for SQL syntax, database constraints, relationships, transactions, and MySQL Connector/J.
- **MDN Web Docs** – Reference material for HTML, CSS, and JavaScript used in web application development.
- **Course Notes and Laboratory Material** – *DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CSI302E)*.
- **Smart Restaurant Ordering System Project Documentation** – Project design, database structure, implementation, testing, deployment, and future-enhancement details.
- **Note:** The submitted documentation recommends removing any reference that was not actually used and adding the exact URLs, books, or tutorials consulted by the team before final submission.

CHAPTER 14 - APPENDICES

Appendix A - Proposed MySQL Schema

The following SQL is a proposed schema consistent with the abstract. It is not extracted from the team's actual database and should be replaced if the implemented schema uses different names or fields.

```
CREATE DATABASE IF NOT EXISTS restaurant_db;
USE restaurant_db;

CREATE TABLE customer (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    email VARCHAR(120)
);

CREATE TABLE menu_item (
    item_id INT PRIMARY KEY AUTO_INCREMENT,
    item_name VARCHAR(120) NOT NULL,
    category VARCHAR(80),
    price DECIMAL(10,2) NOT NULL CHECK (price >= 0),
    availability BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(30) NOT NULL DEFAULT 'PLACED',
    total_amount DECIMAL(10,2) NOT NULL DEFAULT 0,
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
);

CREATE TABLE order_item (
    order_item_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT NOT NULL,
    item_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    line_total DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (item_id) REFERENCES menu_item(item_id)
);

CREATE TABLE bill (
    bill_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT NOT NULL UNIQUE,
    bill_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    amount DECIMAL(10,2) NOT NULL,
    payment_status VARCHAR(30) DEFAULT 'PENDING',
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);
```

Appendix B - Proposed Order Transaction Pattern

```
Connection con = dataSource.getConnection();
try {
    con.setAutoCommit(false);

    // 1. Insert order header and obtain generated order_id.
```



```
// 2. For each selected item, read current price from menu_item.  
// 3. Insert order_item with quantity and line_total.  
// 4. Update orders.total_amount.  
// 5. Insert bill record.  
  
con.commit();  
} catch (Exception ex) {  
    con.rollback();  
    throw ex;  
} finally {  
    con.close();  
}
```

Appendix C - User Guide

Customer Flow

9. Open the application and view the available menu.
10. Select one or more menu items.
11. Enter the required quantity for each selected item.
12. Submit the order.
13. Review the calculated bill or confirmation shown by the application.

Staff Flow

14. Open the staff/menu management page.
15. Add or update menu item details as permitted by the implementation.
16. View current orders.
17. Update the order status as preparation progresses.
18. Verify stored order and billing records in the application or database tool.

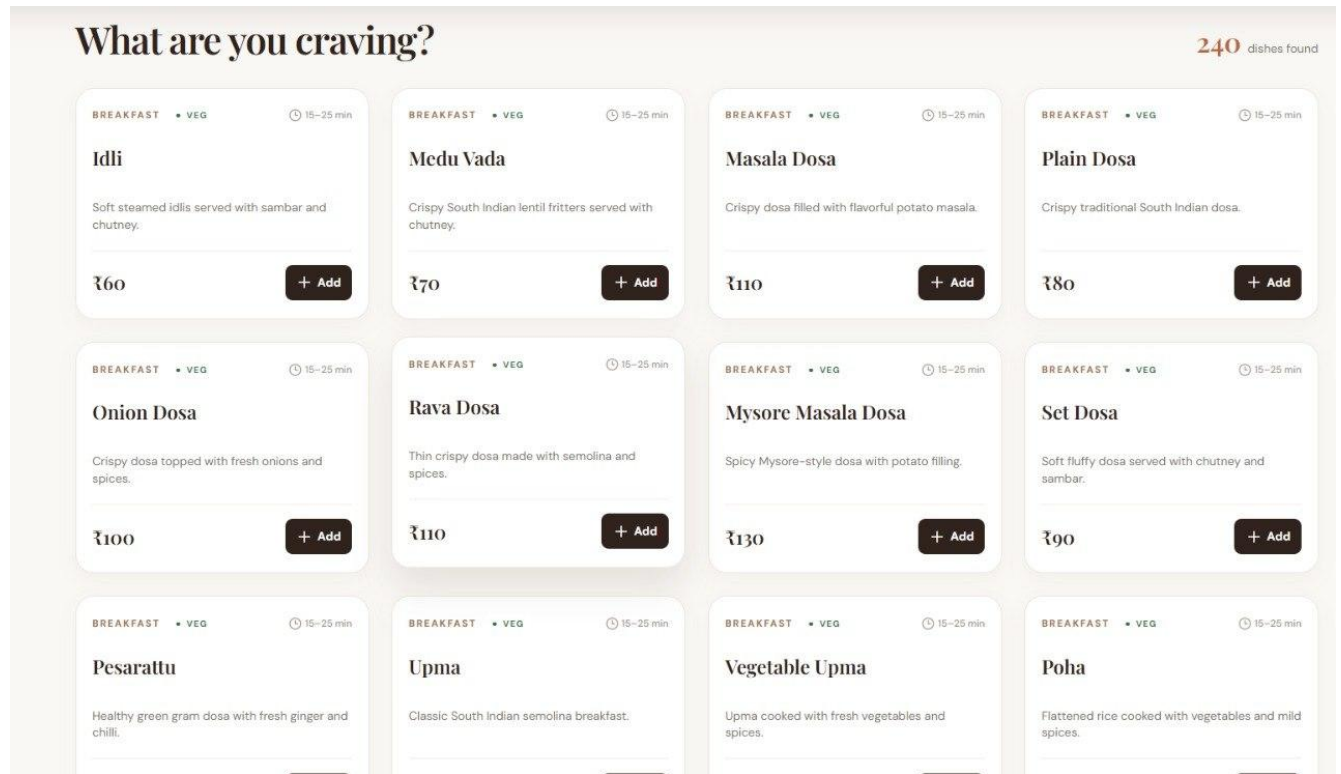
Appendix D - Screenshots / Geo-Tagged Photos / Review Forms

The uploaded project abstract did not include application screenshots, geo-tagged photos, or signed review forms. Insert the actual evidence required by the course in this appendix before final submission. Suggested placeholders:

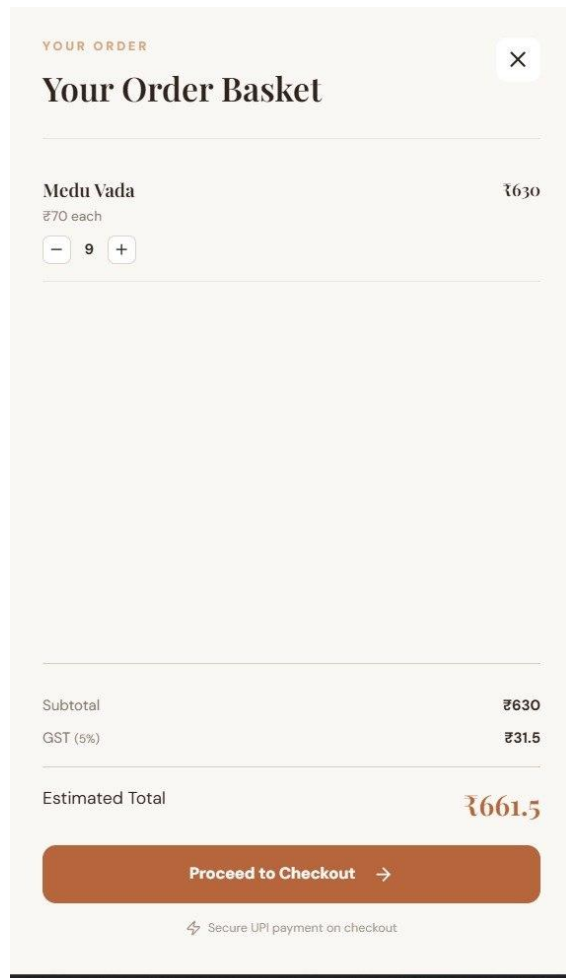
- Screenshot 1 - Home/Menu page



- Screenshot 2 - Order selection page



Screenshot 3 - Order confirmation / bill



Screenshot 6 - MySQL Workbench tables/data

Smart Restaurant Ordering System - Project Documentation

```
viraham-restaurant  database > seed.sql
> .manus-longs
> client
> database
  > mysql-data
  > schema.sql
  > seed.sql
  > dist
> node_modules
> patches
> server
> services
  > menu-service
  > JS server.js
  > order-service
  > JS server.js
> shared
> .env
> .env.example
> .gitignore
> .gitkeep
> .prettierrc
> .prettierrc
> components.json
> package-lock.json
> package.json
> pnpm-lock.yaml
> spice-garden-restaurant.zip
> template.json
> tsconfig.json
> tsconfig.node.json
> vite.config.ts

3
4 -- Insert categories
5 INSERT INTO `menu_categories` (`category_id`, `category_name`, `display_order`) VALUES
6 (1, 'Breakfast', 1),
7 (2, 'Lunch', 2),
8 (3, 'Snacks', 3),
9 (4, 'Dinner', 4),
10 (5, 'Drinks', 5),
11 (6, 'Sweets', 6)
12 ON DUPLICATE KEY UPDATE `category_name`=VALUES(`category_name`);
13
14 -- Insert menu items
15 INSERT INTO `menu_items` (`item_id`, `category_id`, `name`, `description`, `price`, `food_type`,
16 (1, 1, 'Idli', 'Soft steamed idlis served with sambar and chutney.', 60, 'VEG', 'BREAKFAST', 1),
17 (2, 1, 'Medu Vada', 'Crispy South Indian lentil fritters served with chutney.', 70, 'VEG', 'BREAKFAST', 1),
18 (3, 1, 'Masala Dosa', 'Crispy dosa filled with flavorful potato masala.', 110, 'VEG', 'BREAKFAST', 1),
19 (4, 1, 'Plain Dosa', 'Crispy traditional South Indian dosa.', 80, 'VEG', 'BREAKFAST', 1),
20 (5, 1, 'Onion Dosa', 'Crispy dosa topped with fresh onions and spices.', 100, 'VEG', 'BREAKFAST', 1),
21 (6, 1, 'Rava Dosa', 'Thin crispy dosa made with semolina and spices.', 110, 'VEG', 'BREAKFAST', 1),
22 (7, 1, 'Mysore Masala Dosa', 'Spicy Mysore-style dosa with potato filling.', 130, 'VEG', 'BREAKFAST', 1),
23 (8, 1, 'Set Dosa', 'Soft fluffy dosa served with chutney and sambar.', 90, 'VEG', 'BREAKFAST', 1),
24 (9, 1, 'Pesarattu', 'Healthy green gram dosa with fresh ginger and chilli.', 100, 'VEG', 'BREAKFAST', 1),
25 (10, 1, 'Upma', 'Classic South Indian semolina breakfast.', 60, 'VEG', 'BREAKFAST', 1),
26 (11, 1, 'Vegetable Upma', 'Upma cooked with fresh vegetables and spices.', 75, 'VEG', 'BREAKFAST', 1),
27 (12, 1, 'Poha', 'Flattened rice cooked with vegetables and mild spices.', 65, 'VEG', 'BREAKFAST', 1),
28 (13, 1, 'Pongal', 'Creamy rice and lentil dish tempered with spices.', 80, 'VEG', 'BREAKFAST', 1),
29 (14, 1, 'Ven Pongal', 'Traditional South Indian savory pongal.', 90, 'VEG', 'BREAKFAST', 1),
30 (15, 1, 'Poori with Potato Curry', 'Fluffy fried pooris served with potato curry.', 100, 'VEG', 'BREAKFAST', 1),
31 (16, 1, 'Chapati with Veg Kurma', 'Soft chapati served with creamy vegetable kurma.', 110, 'VEG', 'BREAKFAST', 1),
32 (17, 1, 'Aloo Paratha', 'Stuffed Indian flatbread filled with seasoned potato.', 110, 'VEG', 'BREAKFAST', 1),
33 (18, 1, 'Paneer Paratha', 'Stuffed paratha filled with spiced paneer.', 130, 'VEG', 'BREAKFAST', 1),
34 (19, 1, 'Sabudana Khichdi', 'Tapioca pearls cooked with peanuts and mild spices.', 90, 'VEG', 'BREAKFAST', 1),
35 (20, 1, 'Moong dal Chilla', 'Healthy savory pancakes made from green gram.', 100, 'VEG', 'BREAKFAST', 1),
36 (21, 2, 'Veg Meals', 'Complete vegetarian meal with rice, curry and sides.', 180, 'VEG', 'LUNCH', 1),
37 (22, 2, 'Andhra Veg Meals', 'Traditional Andhra vegetarian meal with flavorful sides.', 200, 'VEG', 'LUNCH', 1),
38 (23, 2, 'South Indian Thali', 'Traditional South Indian thali with multiple dishes.', 220, 'VEG', 'LUNCH', 1)
```

- Geo-tagged project-team photo(s), if required
- Guide review form(s) with signatures, if required

(List the frameworks, libraries, platforms, or tools you plan to use.)

Frontend: HTML, CSS, JavaScript
Backend : Java
Database: MySQL
Database Connectivity: JDBC
Development Environment: Visual Studio Code / IntelliJ IDEA
Database Tool: MySQL Workbench

Guide Name and Signature (remarks if any) _____